

INF1103: Programming Fundamentals with DevOps

Week 9: Object Oriented Relationships

Project Updates

- Time for ***project deadline (wk12)*** and ***quiz 2 (wk13)*** are confirmed and available on XSITE Announcements
- Project Brief, Rubric, Report Template, etc. all on XSITE
- Project submission requires an export of Task Management List and timelines from TeamAId, this ***WILL BE*** ready by the time you need to submit it! (will keep you posted)
  - Lots of people used task management really well in first half, now we expect it! Part of report.

Project Updates

- Reminder
  - The most important thing about your project is
    - Showcase Object Oriented Programming
    - Showcase good programming practices
    - Showcase your ability to write good code
  - AI is just a feature, meaningfully implementing AI doesn't mean it has to be complicated.
    - KISS Principle

Discussion Points From Lecture chat (this page will not be tested! ☺)

- Mutable vs Immutable Objects
  - Default is mutable, can change content
  - Immutable means object is locked
  - Won’t worry about this in this module, this is not encapsulation

- RUST
  - Bit of a different paradigm, supports OOP, doesn’t like inheritance, is very fast and is very strict
  - Composition vs Inheritance (will make sense after todays lesson)
  - Video games went this route a long time ago!
  - Can do this in any OOP language!

Discussion Points (may be tested)

- self.__price vs self._price (private vs protected, but not enforced by python) # naming convention and to make you think about software design
- self.__price results in ***name mangling*** for external code:
  - product._Product__price (sort of hides access but can still access)
  - Internally in same class self.__price is ok

- BUT what if you wanted to make life easier for the user so they could just use:
- product.price=50
  - not product.price(50)
  - not product._Product__price = 50
- Create price as a property!

Learning Outcomes

- More Encapsulation (@Property)
- Inheritance (with super and overriding)
- Abstract classes
- Interfaces and Multiple Inheritance

More Encapsulation (@Property)

- @property and @propertyname.setter

Code inside Product Class:

```
@property
def price(self):        # Defines the property
    return self.__price

@price.setter
def price(self, value):   # Adds a setter to that property
    self.__price = value
```

States we are about to create a property

Line after @property is our property, so 'price' is now treated as a property

Controls what we return

Controls how we set

Using the property 'price' we now do @price.setter to allow value to be set.

External code:

```
product.price      # Gets the price value returned by def price(self)
product.price = 20  # Set the property @price via def price(self, value)
```

Note: @property changes how def price(self) works, trying to call it as a method will no longer work
So, product.price() won't work after @property is used to convert that method to a property

Inheritance

- **Classes and Objects Recap**
  - A **class** is a blueprint, it **defines** the **attributes** (member variables) and **behaviours** (methods) of an object
  - A **class** should have a **singular responsibility** (SRP)
  - **Objects are instantiations** of the class, the ‘thing’ that we can use in our program
  - We **can create many objects** from the **same class**, all operate on their own data and use their own methods
  - When we create an object the __init__ initializer (constructor) is called where we can set initial values

Inheritance

- **Inheritance**
  - Inheritance is a way in which a class can take **shared behaviours and attributes** from its parent (Good for DRY)
  - This means, if the parent has the attribute ‘name’, then the child will also have the attribute ‘name’, if the parent has a method ‘printdata()’ then the child will also have that method
  - A child can access any attribute or method that a parent has (in python), but we can consider encapsulation conventions:
    - __name # closest to true private, meaning child has it but shouldn’t access
    - _name # protected, meaning child has and can access but external code can’t
    - name # considered a public variable
  - A **child is** considered a **specialised** or **enhanced** version of a parent, a child has everything the parent has and more added on

Inheritance

- **Why Inheritance**
  - If we know we will have different **objects that have a lot of similarities**, Inheritance means we don't have to repeat attributes / methods in many classes (DRY)
  - **Inheritance also allows** for another very cool concept called **Polymorphism** which we will cover in later sessions
  - We can also make a **change to a parent which will impact all of the children**, without having to make the same change in all of the children

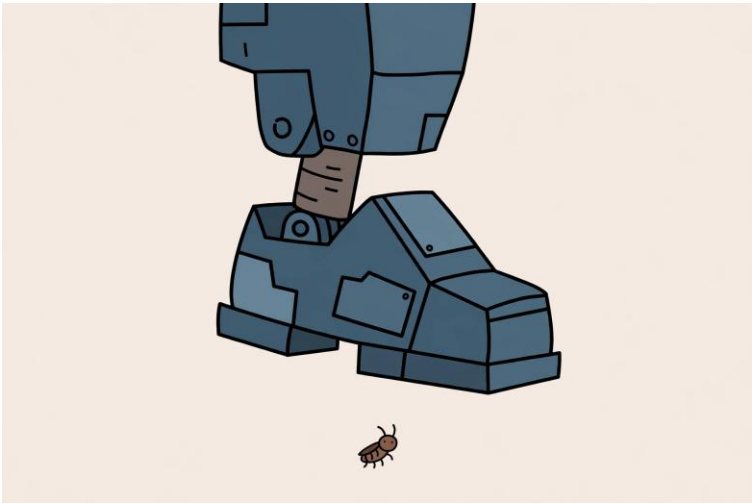
Notes:

- ***Parents are considered 'super' classes and children are considered 'sub' classes.***
- ***Inheritance is considered an IS-A relationship, very important!***

Inheritance

- When to use inheritance, **is-a!**
  - A computer has a SSD, GPU, CPU, Motherboard, PSU
    - A desktop machine **is-a** computer (has all of the above)
    - A laptop **is-a** computer (has all of the above but in different format)
  - A terminator **is-a** robot
  - A dog **is-a** animal, a cat **is-a** animal
  - A cockroach **is-a** bug, but must be terminated ☺

- This is based on a relationship, so a dog **is-a** animal, but it has its own set of features that distinguish it from other animals



Inheritance

- Draw an Animal



What the ...!

Why did we get this? (Should it exist?)
More on this later!

Specifics

More Specifics













Inheritance

Robot Class:

```
class Robot:
    def __init__(self, name, energy):
        self._name = name
        self._energy = energy

    def move(self):
        print(f"{self._name} is moving!")
```

Child Robot Class:

```
class ChildRobot(Robot):
    def __init__(self, name, energy, laser_power):
        super().__init__(name, energy)
        self._laser_power = laser_power

    def shoot_lasers_from_eyes(self):
        print(f"{self._name} shoots lasers at power self._laser_power}!")
```

External Code:

```
# Create a robot
robot = Robot("Robo", 100)
robot.move()

# Create a child robot
child = ChildRobot("LaserBot", 80, 50)
child.move()                    # Inherited method
child.shoot_lasers_from_eyes()  # Child's own method
```

Notes:

- Robot is the parent class, it has attributes name and energy
- ChildRobot Inherits from RobotClass, it adds attributes laser_power and a method called shoot_lasers_from_eyes
- External Code creates a basic robot (robot) and a more advanced robot (child) and calls their methods

Inheritance

- Note, we can not access a child method from a parent class
  - *Robot.shoot_lasters_from_eyes() # invalid*
- By default a child calls its own __init__ but the parent may have useful __init__ stuff too!
  - *We use **super()** to call parent methods directly*
- We can however call a parent method from a child class
  - child.move() # valid
- But, what if the child moves differently from the parent?

External Code:

```
# Create a robot
robot = Robot("Robo", 100)
robot.move()

# Create a child robot
child = ChildRobot("LaserBot", 80, 50)
child.move()                  # Inherited method
child.shoot_lasers_from_eyes()   # Child's own method
```

```
class ChildRobot(Robot):
    def __init__(self, name, energy, laser_power):
        super().__init__(name, energy)
```

Inheritance [Overriding]

External Code:

```
robot = Robot("Robo", 100)
robot.move()  # Robo is using wheels to move.

child = ChildRobot("LaserBot", 80, 50)
child.move()  # LaserBot is using legs to move.
```

- *Notice both child and parent called move()*
- *The parent has no knowledge of child*
- *Parent will invoke 'is using my wheels to move'*
- *Child has knowledge of parent but can see it also has a move method, so will use its own version, child uses 'is using my legs to move'*
- *If parent move is useful to child, child can call parent move first super.move()*

```
class Robot:
    def __init__(self, name, energy):
        self._name = name
        self._energy = energy

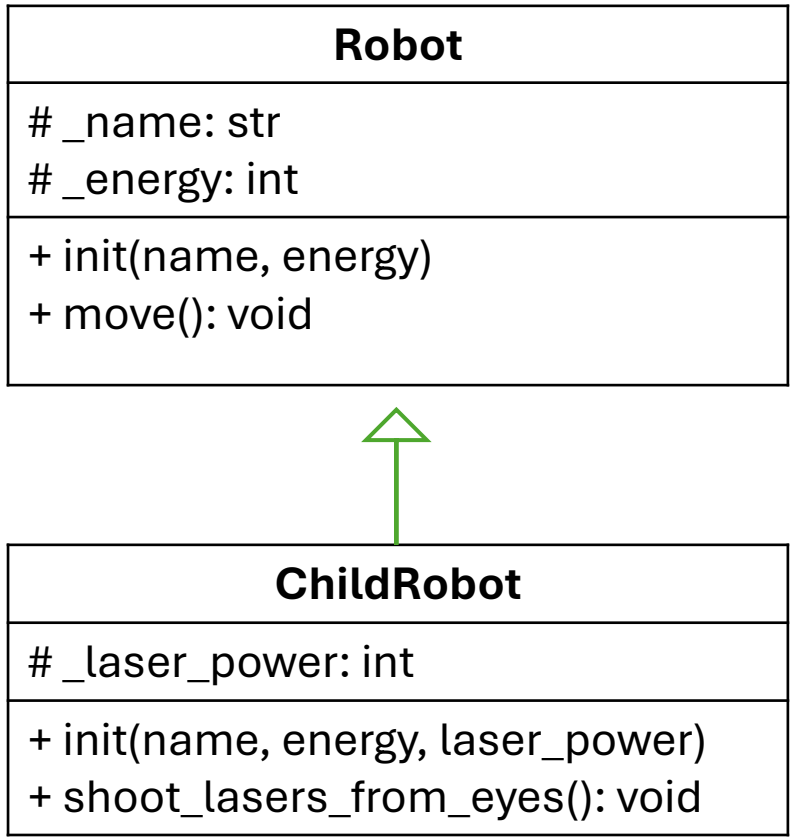
    def move(self):
        print(f"{self._name} is using my wheels to move.")
```

```
class ChildRobot(Robot):
    def __init__(self, name, energy, laser_power):
        super().__init__(name, energy)
        self._laser_power = laser_power

    def move(self):  # Overriding the parent method
        print(f"{self._name} is using my legs to move.")

    def shoot_lasers_from_eyes(self):
        print(f"{self._name} shoots lasers at power {self._laser_power}!")
```

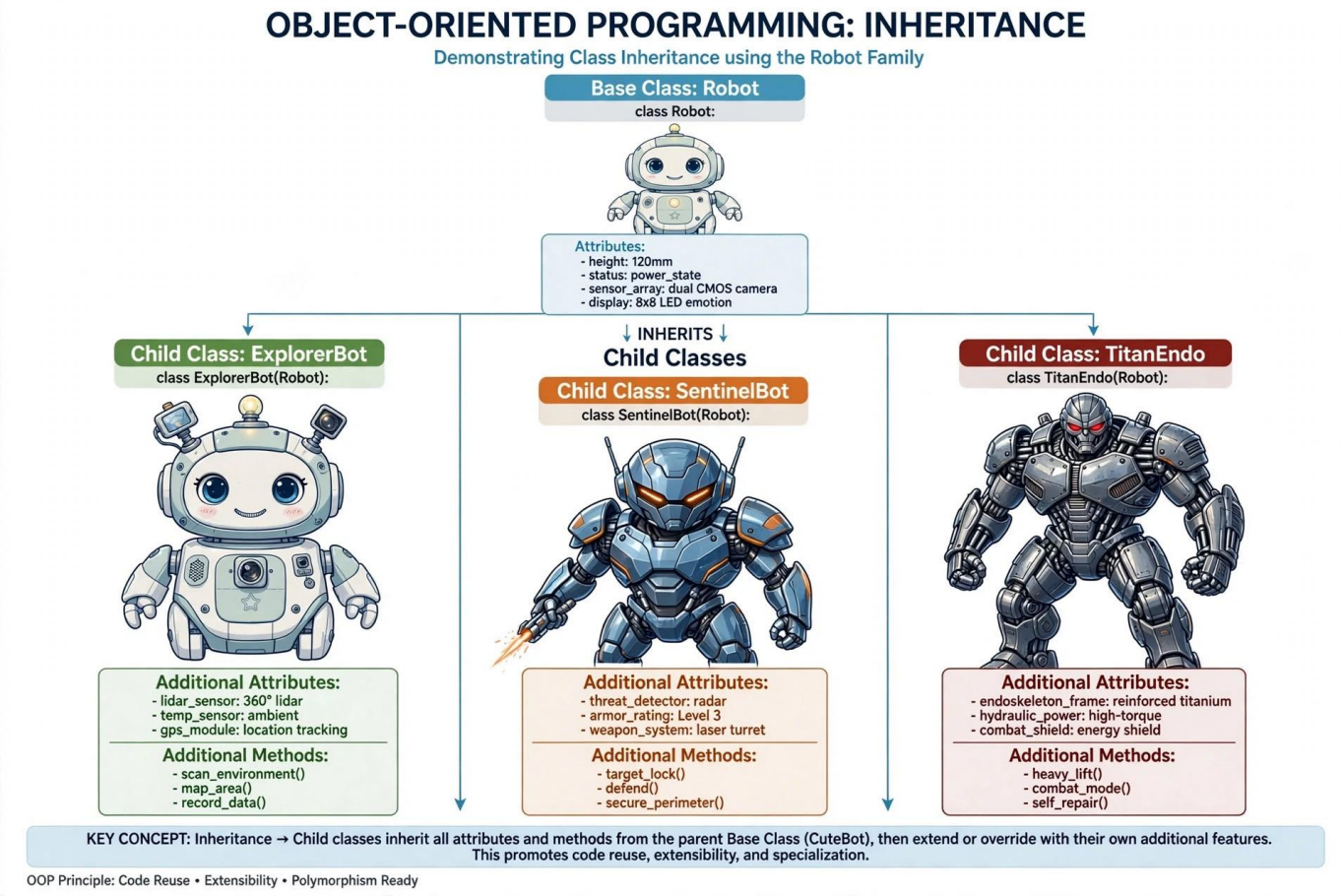
Inheritance



Hollow Arrow from child to parent represents Inheritance

Inheritance

- Our Robot Army is complete!
- But what if we think Robot is kind of rubbish?
  - Doesn't have enough features to be useful
  - Does collect all the shared features of the other bots
  - Should we just copy the attributes/methods across other bots?



Abstract Classes

- Reminder (DRY): In programming, if you are seeing code repeated, you're doing something wrong!
- If we think Robot is **not good enough to ever be an object by itself**, we can turn it into an **abstract class**
- An abstract class is a class that defines the core of an object, but **doesn't have enough detail** to be an actual object by itself
- An **abstract** class can **not be instantiated**
- Animal vs Cat / Dog / Monkey



Oh no! It's back! (It shouldn't exist!!!!)

Abstract Classes

- ABC = abstract class
- class Robot(ABC) defines Robot as an Abstract class
- This simply means, it can't be instantiated, a design choice!
- pass means we don't want to implement anything and the child will take care of implementation
- WheelRobot is a concrete class, this can be instantiated and inherits from the abstract class Robot

```
from abc import ABC, abstractmethod

class Robot(ABC):
    @abstractmethod
    def move(self):
        pass


class WheelRobot(Robot):
    def move(self):
        print("Using my wheels to move")

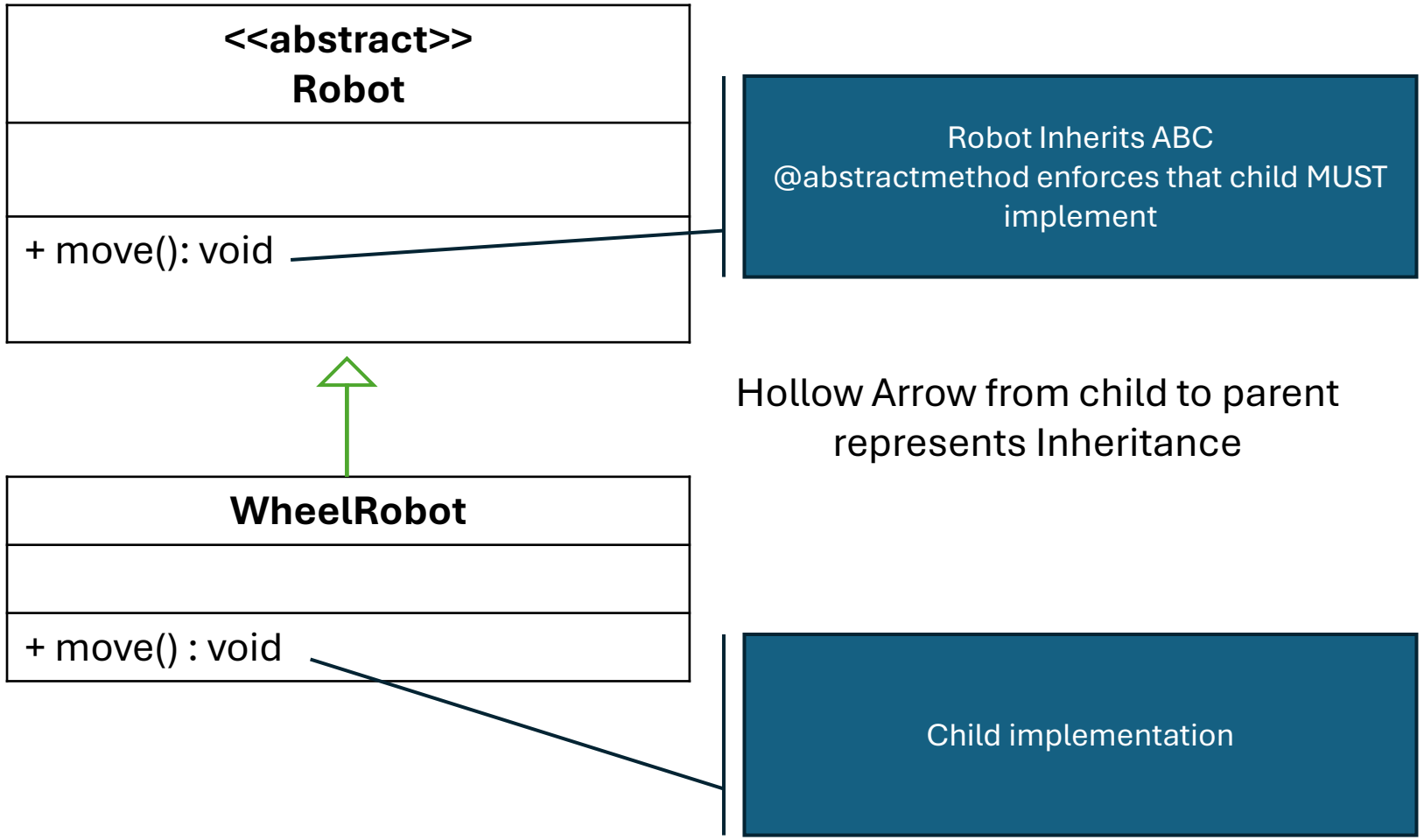
~~robot = Robot()  # TypeError: Cannot instantiate abstract class~~
robot = WheelRobot()
robot.move()
```

Robot Inherits ABC
@abstractmethod enforces that child MUST implement

Must inherit ABC and have @abstractmethod to prevent object instantiation (correct abstract class behaviour)

Inheritance

- ***Do we need to show parent methods in child?:***
  - ***Inherited, unchanged method? don't repeat it.***
  - ***Overridden method, show it in the child to make the specialised implementation clear.***
  - ***New method or attribute, of course show it in the child.***



Inheritance

- ***WARNING: Do NOT do multiple inheritance***
  - ***(no multiple is-a relationships)***
  - ***Python allows it, many other languages do not***
  - ***Leads to very confusing code and relationships***
  - ***(poor understandability and readability)***

- Python allows it, some languages do not, but it can be avoided and is best to avoid

Inheritance

- **Multiple Inheritance pitfalls**
  - FancyRobot inherits from Wheeler and Walker
  - Our object stored in fancyrobot calls move()
  - Walker and Wheeler are at the same ‘parent’ level, they are not related to each other, only FancyRobot is related to both
  - Python will actually just call the first in the inheritance list, so ‘walking on legs’
  - Unexpected behaviour: A developer might expect the robot to roll on wheels, but it walks instead.

```
class Walker:
    def move(self):
        print("Walking on legs")

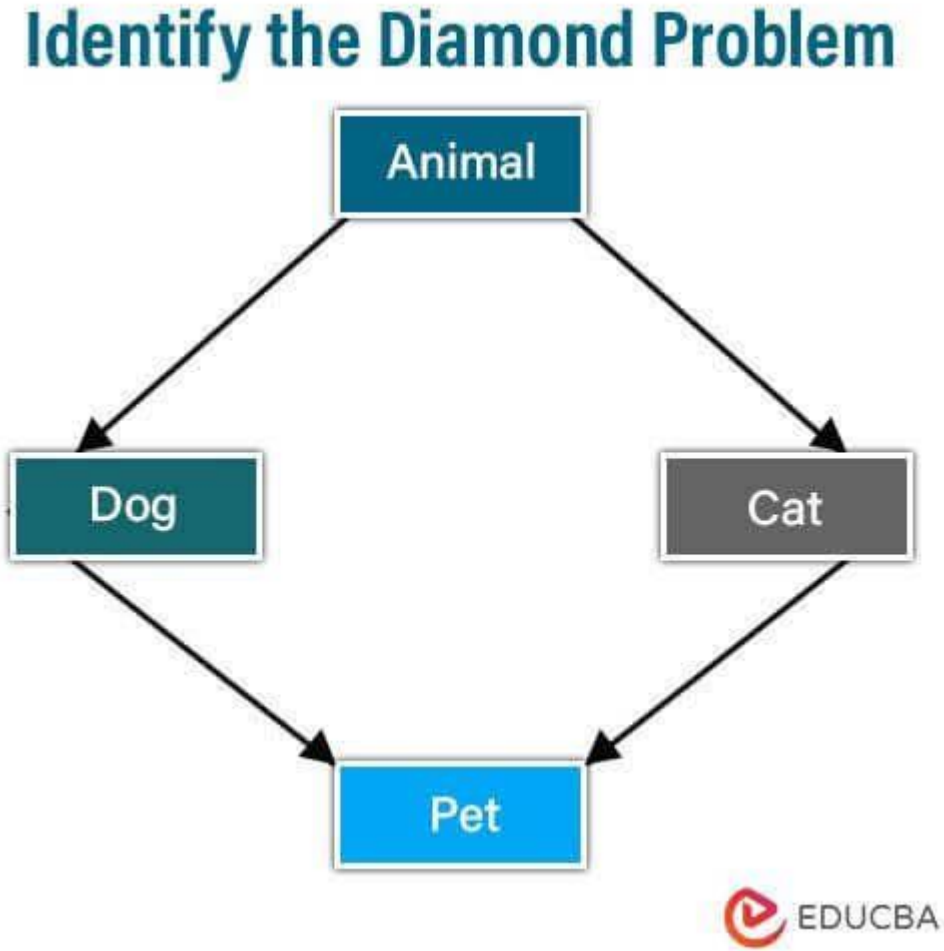

class Wheeler:
    def move(self):
        print("Rolling on wheels")


class FancyRobot(Walker, Wheeler): #inherits both
    pass


fancyrobot = FancyRobot()
fancyrobot.move()
```

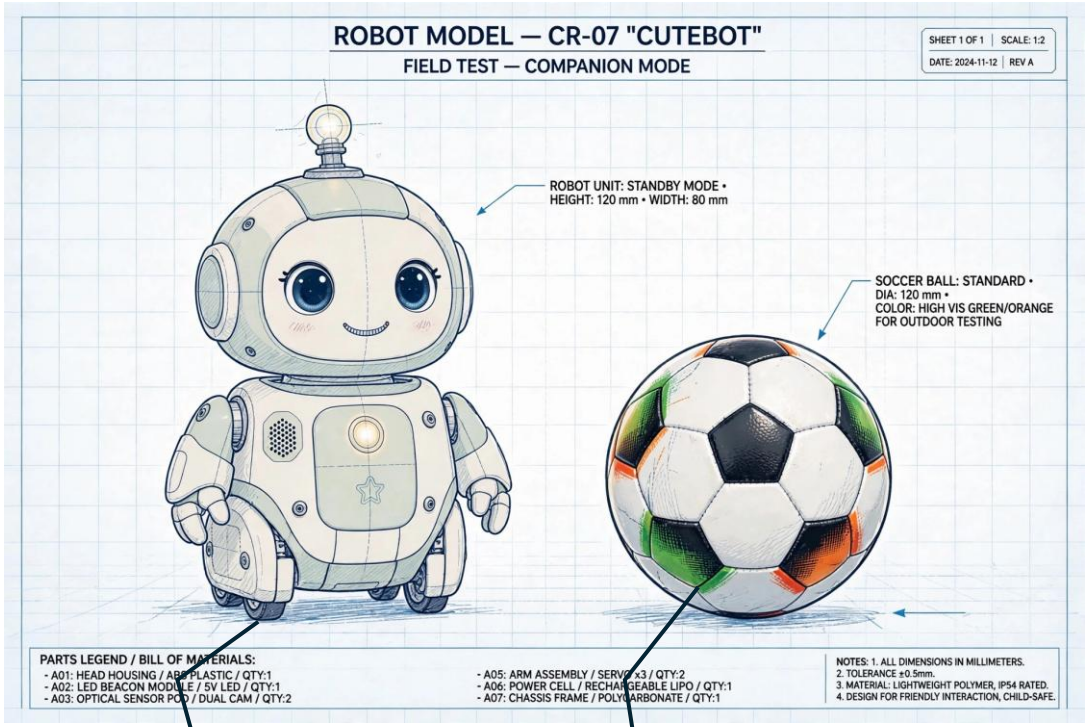
Inheritance

- **Multiple Inheritance pitfalls**
  - Diamond problem: Two parent classes inherit from the same grandparent. Which implementation gets called? Python uses its Method Resolution Order (MRO) to decide.
  - If we ask Pet to speak, does it bark or meow?
  - Depends on the order in which you inherited Dog / Cat or Cat / Dog in the Pet class!
  - Confusing, best to avoid (very possible to avoid!)



Interfaces

- **Relationships can be functional** as well, to form this relationship we use **interfaces**
- **Inheritance:** Defines an **‘is-a’** relationship, shares attributes and methods
- **Interface:** Defines what a class **‘must be able to do’** (e.g. move) even if those classes don’t have an is-a relationship



They both move differently
A robot is-a ball? Nope
A ball is-a robot? Nope

Interfaces

- Interfaces are like Contracts
  - **Must be able to do**
  - If a **class implements** an **interface**, it is **promising** it will **be able to do** some function
  - **Different objects** may have the same required capability but **different implementations**.
  - The contract states what all of those objects must be able to do, how they do it is up to them
  - Example payable
    - Credit Card, Bank Transfer, PayPal all can implement Payable interface
    - A promise that they all have the ability to pay
    - How each object pays (implementation) is up to them



Interfaces

- Python doesn't have a keyword for interfaces like other languages
- BUT, python does allow interface style functionality with the same ABC process
- We can use multiple ABC if
  - only one is inheritance (based on is-a relationship0
  - Other(s) is/are interfaces (based on 'must be able to do' relationship
- Multiple ***interface*** ABC is fine
- This is a relationship difference

- *Note: Reminder, other languages don't use ABC for interfaces, they have a separate way to make interfaces*

Interfaces

- Movable uses ABC but it is a functional relationship (not is-a) so an interface
- We could decide that it is not only robots that need to move() so put move() into an interface
- Reminder: Other languages have specific syntax for interfaces
- For WheelRobot, Robot is inheritance (is-a) where Movable and Rechargeable are functional (not is-a)
- This example is good, only 1 inheritance and multiple interfaces

```
from abc import ABC, abstractmethod

class Movable(ABC):
    @abstractmethod
    def move(self):
        pass
```

```
class WheelRobot(Robot, Movable, Rechargeable):
    def move(self):
        print(f"{self._name} is using my wheels to move")
```

Question, implement interface in Robot or WheelRobot?

Interfaces

- Interfaces are functional
  - A cat can speak
  - A dog can speak
  - CanSpeak() interface
  - CanSpeak() can be an abstract method, child must implement
  - Cat implements CanSpeak()
  - Dog implements CanSpeak()
  - Cat meows, Dog Barks

```
from abc import ABC, abstractmethod

class CanSpeak(ABC):
    @abstractmethod
    def speak(self):
        pass
```

```
class Cat(CanSpeak):
    def speak(self):
        print("Meow!")
```

```
class Dog(CanSpeak):
    def speak(self):
        print("Woof!")
```

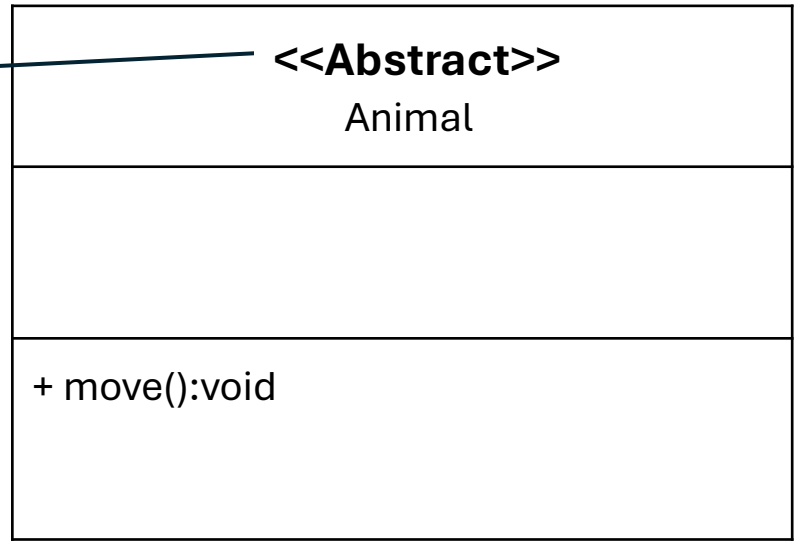
```
cat = Cat()
dog = Dog()

cat.speak()  # Meow!
dog.speak()  # Woof
```

Interfaces [Worked Example]

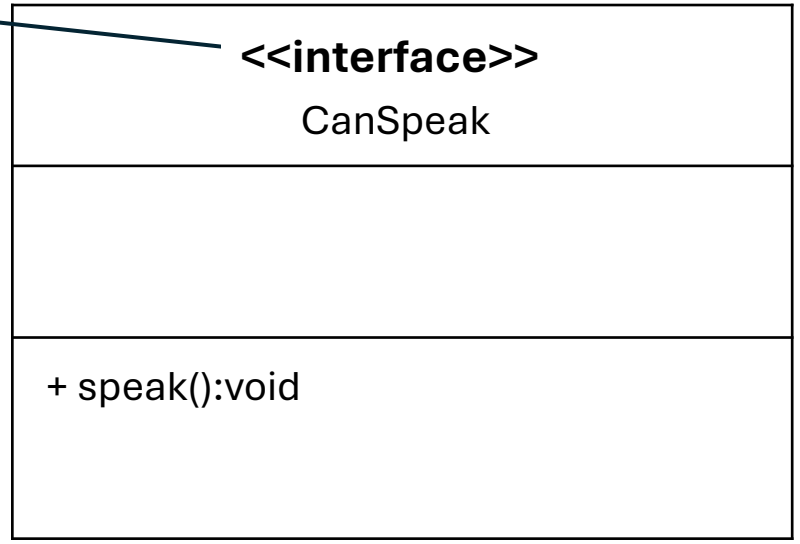
```
class  Animal(ABC):
    @abstractmethod
    def move(self):
        pass
```

We decided it is abstract based on is-a relationship




```
class CanSpeak(ABC):
    @abstractmethod
    def speak(self):
        pass
```

We decided it is interface based on must-be-able-to-do relationship



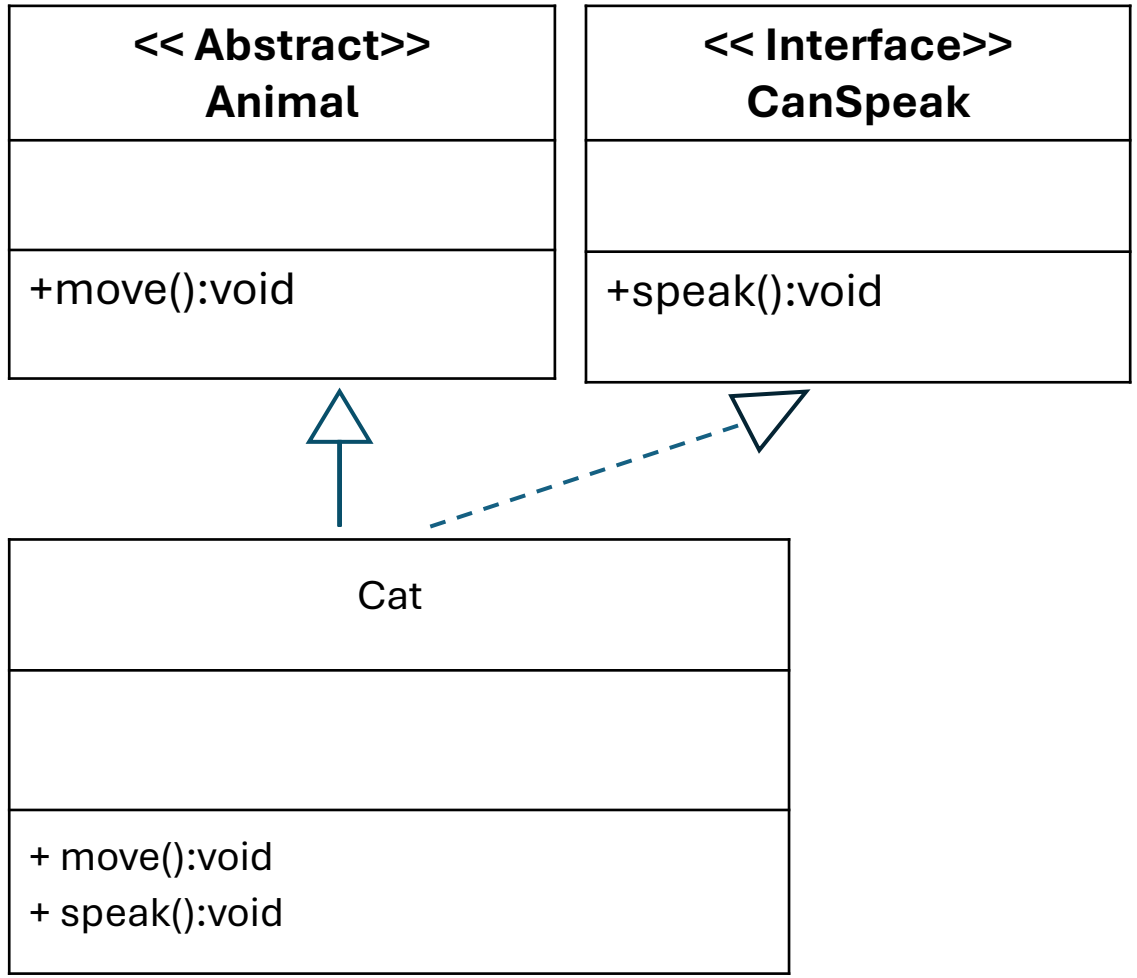
Reminder : Python declaration is the same but other languages have unique ways to declare interfaces

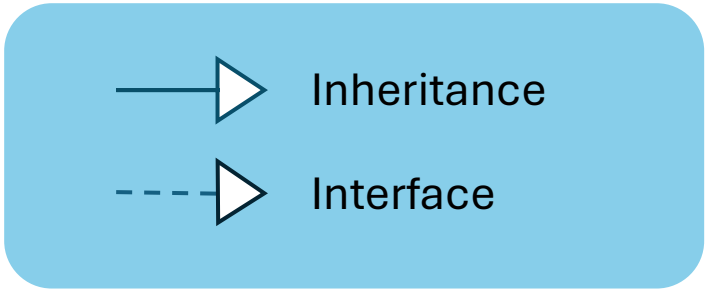
Interfaces [Worked Example]

```
class Cat (Animal, CanSpeak ):

    def move(self):
        print("Cat walks")

    def speak(self):
        print("Meow!")
```





Interfaces [Worked Example]

```
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def move(self):
        pass
```

```
class CanSpeak(ABC):
    @abstractmethod
    def speak(self):
        pass
```

```
class Cat(Animal, CanSpeak):
    def move(self):
        print("Cat walks")

    def speak(self):
        print("Meow!")
```

```
Class Dog(Animal, CanSpeak):
    def move(self):
        print("Dog walks")

    def speak(self):
        print("Woof!")
```

External Code:

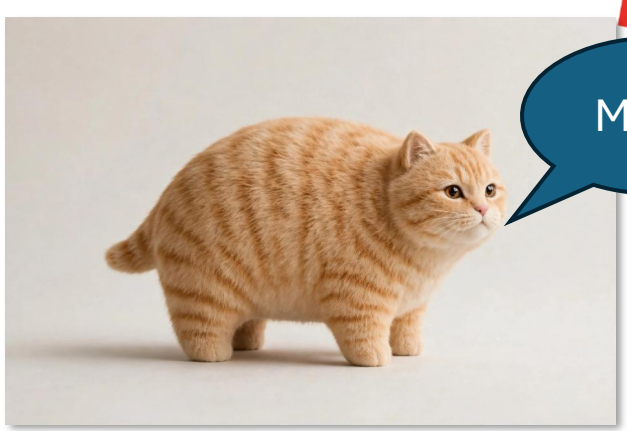
```
cat = Cat()
dog = Dog()

cat.speak()  # Meow!
dog.speak()  # Woof!
```

Abstract ensures all animals can move

Interface ensures anything that says it can speak, can speak

Both enforce structure and can reduce unexpected runtime errors





Remember

- Behind Every Cute Cat



Is this monstrosity

S.O.L.I.D

- The letter for today is 

  - The Open Close Principle:
  - Software entities (such as classes, modules, and functions) should be open for extension but closed for modification
  - Design the system so new types can be added without changing the code that uses them.

```
def make_animal_speak(animal):
    if animal == "cat":
        print("Meow!")
    elif animal == "dog":
        print("Woof!")
```

VS

```
class Cat:
    def speak(self):
        print("Meow!")

class Dog:
    def speak(self):
        print("Woof!")

def make_animal_speak(animal):
    animal.speak()
```

Achievable with : Inheritance / Interfaces / ABstract Classes / Composition

What's Next?

- More relationships
  - Association / Aggregation / Composition / Polymorphism
- Lab Activity
  - Last week lab due Friday 23:59
  - This weeks lab due next Friday 23:59
- Keep Calm and Carry On

Thanks!!!

